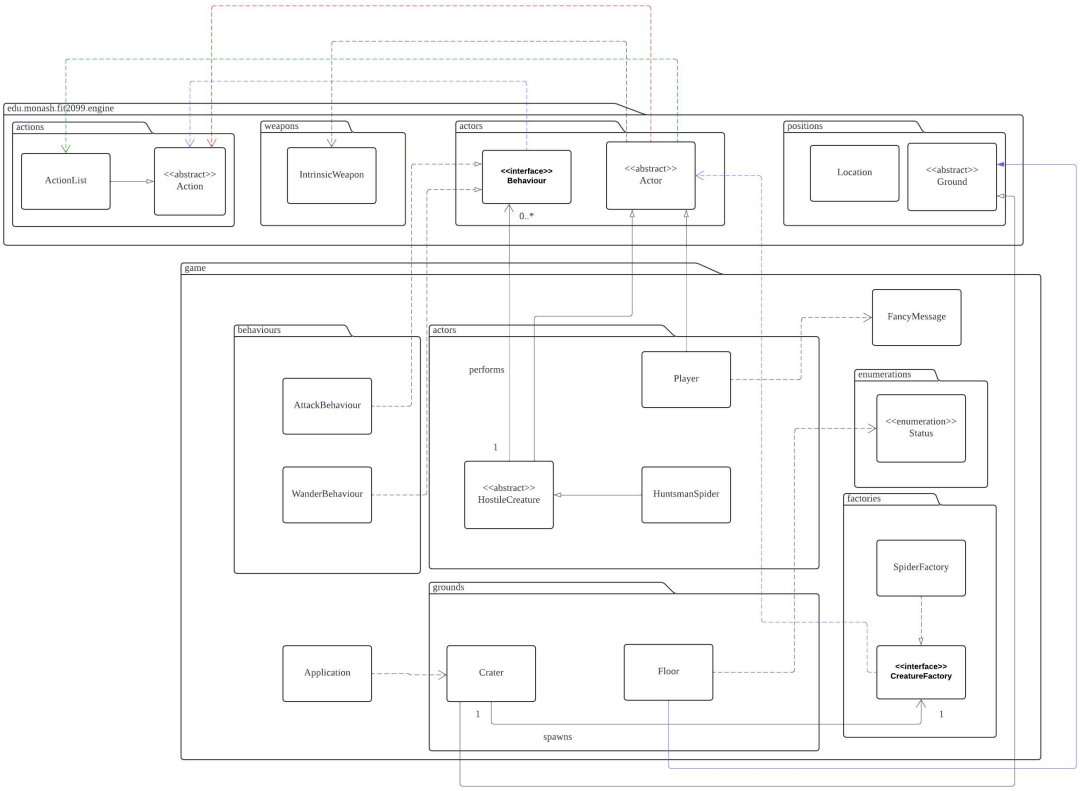


This diagram represents the object-oriented design of Assignment 1 Requirement 3 which aims to create the attack feature of the enemies and a “spawner” which is able to spawns new enemies to the map. The diagram consists of 2 main packages, the game engine provided and the game itself. Inside the engine package, the abstract classes involved are Action, Ground and Actor, the concrete classes involved are ActionList, IntrinsicWeapon and Location. For the interface, we have the Behaviour.

Inside the game package, it consists of two behaviour which inherit the Behaviour interface from engine package. HostileCreature abstract class and Player concrete class which inherit the Actor abstract class in engine package, as well as a HuntsmanSpider class inherits HostileCreature abstract class. Other than that the game package consists of concrete class such as Crater and Floor inheriting Ground abstract class, SpiderFactory class implementing CreatureFactory interface. Lastly it consist of concrete classes Application and FancyMessage, enumeration class such as Status.

The object-oriented design principles applied in this design is:

- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle
- Liskov Substitution Principle
- DRY Principle

How are the principles being applied?

- Each classes have their own responsibility. For example, HostileCreature abstract class represents the general hostile creatures existed in the game and SpiderFactory class is responsible for spawning new spider creatures etc. This design allows each class to focus on a specific aspect of the game. This helps in code maintainability. (Single Responsibility Principle)

- The design allows extension without modifying the existing code. If there is new behaviour or new hostile entities being introduced. We can simply just add the new concrete implementations by using abstraction without altering existing classes which promotes flexibility and code extensibility. (Open-Closed Principle)

- The design encourages usage of abstractions rather than direct concrete implementations which reduce coupling and promote flexibility in the code. It allows generic attributes and characteristics of two concrete classes being shared without displaying direct dependencies between them. (Dependency Inversion Principle)

- The usage of polymorphism is possible in this design. For example, Player and HuntsmanSpider object can be treated as Actor object which is at a higher level class, allowing a uniform handling towards different types of entities in the game. The design enables system to easily accommodate new subclasses and variations without compromising the program. (Liskov Substitution Principle)

- The usage of abstract classes allow generic attributes of concrete classes to be shared among them without inputting the same code. This enhances code efficiency and maintainability. (DRY Principle)

The pros and cons in the system:

Pros:

- Adhere SOLID principles which promotes code readability, flexibility and maintainability.
- Reduced dependencies and coupling between the classes means reduce the risk of bugs occurring when the code is being modified.
- Polymorphism is allowed and able to treat the objects uniformly, enabling interchangeable use of object type.
- Introduction of interface reduces unnecessary dependencies and allow multiple inheritance.

Cons:

- As the system scales bigger, the design can be over complex with the introduction of abstract classes and interfaces.
- It is difficult to maintain large number of classes over time as the design extended.
- Might have potential of tight coupling if concrete classes rely heavily on the implementation details of other classes.

Let's extend the design by introducing a new enemy to the game such as GiantLeech class, which represents a giant leech that does not move but only attack the player nearby. It can be spawned using the same Crater class above. Similar to HuntsmanSpider, the GiantLeech class inherits from the HostileCreature class which already have the AttackBehaviour (the design assumption was all hostile creatures are able to attack players), meaning we do not need to implement the AttackBehaviour inside GiantLeech. For the implementation of the crater spawning GiantLeech object, we need to introduce a factory concrete class name GiantLeechFactory which implements the same interface CreatureFactory as the factory class SpiderFactory which spawns HuntsmanSpider object.

When the Application runs, the crater which has the parameter of GiantLeechFactory will have the chance of spawning new GiantLeech enemies with a certain spawn rate in the game as the game ticks. Once a GiantLeech object spawn nearby the crater, it possessed only the AttackBehaviour unlike the HuntsmanSpider which possessed the WanderBehaviour with the ability to move around, so the GiantLeech is only able to remain in the same coordinates. Once the player is nearby, the GiantLeech is able to attack the player with the specific hit rate using its intrinsic weapon.

Based on the description of the potential extension above, this OOP design is shown to seamlessly integrate the new features by leveraging the inheritance and interface implementation. Although it seems complicated as it requires implementation of two classes, the factory class and the specific creature class itself, it provides the ease of maintaining the code with the minimum risk and it also provides a solid foundation as the game continues to develop and be enhanced.